dow - Chatbot Version Tracking Demo

3-minute product demonstration for **dow - Drift Observation Workbench**

Format: Two people seated at a table. A female developer explains the problem she is facing while working on a restaurant chatbot project. A male developer introduces `dow` as the solution. The first half demonstrates the UI. The second half demonstrates the CLI.

Scenario: The chatbot project is a service chatbot for a Hyderabadi Biryani restaurant. Do not give the chatbot a product name in the video; call it simply "the chatbot" or "chatbot". The codebase used for the demo lives in demo/.

Duration: 3 minutes.

People on camera:

- **Female developer** - working on the chatbot project and struggling to track which prompt/configuration/version produced which behavior.
- **Male developer** - introduces `dow`, first through the UI and then through the CLI.

Timeline

Time	Segment	Main feature
0:00-0:35	Problem introduction	Why chatbot behavior versioning is hard
0:35-1:35	UI demonstration	Versions, tags, drift, stability, eval scores
1:35-2:35	CLI demonstration	`history`, `compare`, `explain`, `eval`, `tree`
2:35-3:00	Wrap-up	What `dow` gives the team

1. Problem Introduction (0:00-0:35)

INT. OFFICE - TABLE SETUP

Two developers are seated at a table with a laptop connected to a shared screen. The screen shows the chatbot project UI. The female developer is looking at a few different chatbot responses and configuration changes.

FEMALE DEVELOPER:
I am working on this service chatbot for a Hyderabadi Biryani restaurant. The chatbot answers customer questions about orders, menu items, pricing, spice level, delivery, and recommendations.

FEMALE DEVELOPER:
The problem is version tracking. I keep changing the system prompt, sampling settings, and evaluators. Some versions give better answers, some versions are more unstable, and after a few experiments I cannot clearly tell which change caused which behavior.

MALE DEVELOPER:
That is exactly the problem `dow` is designed for. It versions the complete AI behavior: prompt, model/provider, sampling settings, inputs, outputs, evaluation metrics, drift, stability, and lineage.

FEMALE DEVELOPER:
So instead of only tracking code, we can track how the chatbot behaves?

MALE DEVELOPER:
Yes. Think of it as version control for AI behavior. Let me show you first in the UI, then we can look at the CLI.

2. UI Demonstration (0:35-1:35)

SCREEN: dow UI - Project Overview

The UI shows a project named **chatbot**. The main panel shows a list of captured versions and summary metrics.

UI CONTENT TO SHOW:

```
Project: chatbot

Versions
v1  baseline ordering assistant          stability 0.95   tag: baseline
v2  warm welcome and quote prices         stability 0.97
v3  confirm spice level                   stability 0.98
v4  recommend special and pairing         stability 0.99   tags: good, golden
v5  high temperature stress test          stability 0.80   tag: bad
v6  deterministic release candidate       stability 1.00   tag: release
```

MALE DEVELOPER:

Here the UI is showing every captured behavior version of the chatbot. Each version represents a complete inference spec, not just a text prompt. `dow` captures the prompt, model/provider, sampling settings, input, outputs, runtime metadata, and evaluations.

FEMALE DEVELOPER:

So v1 is the baseline, v4 is the good version, v5 is the stress test, and v6 is a release candidate.

MALE DEVELOPER:

Correct. The tags make the history easy to read. You can mark a version as `baseline`, `good`, `golden`, `bad`, or `release` and compare against those names later.

SCREEN: UI - Version Details for v4

The UI opens version **v4**.

UI CONTENT TO SHOW:

```
Version: v4
Tags: good, golden
Stability: 0.99
Model provider: python
Model entry point: chatbot.py:reply
Samples: 6

Evaluation
captures_order       1.000
greets               1.000
names_dish           1.000
states_price         1.000
confirms_spice       1.000
recommends_special   1.000
suggests_pairing     1.000
avg_response_length  78.83
```

MALE DEVELOPER:
This version is the golden version. It passes the service checklist: it greets the customer, names the dish, states the price, confirms spice level, recommends the special, suggests a pairing, and still captures the actual order.

FEMALE DEVELOPER:
This is the kind of view I need. I can see not only that v4 exists, but why it is considered good.

SCREEN: UI - Compare v4 and v5

The UI switches to a comparison view between **v4** and **v5**.

UI CONTENT TO SHOW:

```
Compare: v4 -> v5

Changed field
sampling.temperature: 0.2 -> 0.9

Semantic drift: 0.24
Stability: 0.99 -> 0.80
Verdict: Behavior Drift

Evaluation changes
captures_order:      1.000 -> 1.000
avg_response_length: 78.83 -> 78.00
```

MALE DEVELOPER:
Here we compare the good version to the high-temperature stress test. The UI shows the exact changed field, the drift score, the stability drop, and the verdict. In this case, the chatbot still captures the order, but its behavior is less stable because temperature was raised.

FEMALE DEVELOPER:
So I do not need to manually inspect every response and guess. The UI gives me a behavior history.

MALE DEVELOPER:
Exactly. Now let us see the same information from the CLI.

3. CLI Demonstration (1:35-2:35)

SCREEN: Terminal

The male developer opens a terminal in the demo project.

MALE DEVELOPER:
The demo code is under demo/. It uses a local Python provider, `chatbot.py:reply` , and custom evaluators in `evals.py` . That means we can run the whole demo offline.

SCREEN (`python demo/run_demo.py --pause 1`):

```
v1 - baseline: a plain ordering assistant
Captured v1   stability 0.950

v2 - prompt edit: warm welcome + always quote prices
Captured v2   stability 0.970

v3 - prompt edit: confirm the spice level on every order
Captured v3   stability 0.978

v4 - prompt edit: recommend the signature dish + suggest a pairing
Captured v4   stability 0.988

v5 - stress test: raise temperature 0.2 -> 0.9
Captured v5   stability 0.801

v6 - branch from v4: pin temperature 0.0 for a deterministic release
Captured v6   stability 1.000
```

MALE DEVELOPER:

This script evolves the chatbot through multiple versions. Each `dow run` captures the current behavior and automatically evaluates it.

FEMALE DEVELOPER:

And these are the same versions we saw in the UI.

MALE DEVELOPER:

Yes. Now I can inspect history from the CLI.

SCREEN (`dow history`):

```
chatbot: behavior history

version   stability   change           tags
v1        0.950       baseline         baseline
v2        0.970       config changed
v3        0.978       config changed
v4        0.988       config changed   good, golden
v5        0.801       config changed   bad
v6        1.000       config changed   release
```

MALE DEVELOPER:

`dow history` gives the behavior timeline. The version ids are automatic, and the tags make important checkpoints easy to reference.

SCREEN (`dow compare v4 v5`):

```
What changed in the configuration

field                  A      B
sampling.temperature   0.2    0.9

signal               value
Output difference    0.470
Semantic drift       0.084  (warn 0.15, fail 0.40)
Stability v4         0.988
Stability v5         0.801

Verdict: Behavior Drift
```

MALE DEVELOPER:

`dow compare` tells us what changed and how much the behavior moved. We can see that only `sampling.temperature` changed between v4 and v5, and the stability dropped.

SCREEN (`dow explain v4 v5`):

```
Cause:   sampling.temperature  (0.2 -> 0.9)
Effect:  semantic drift 0.084, stability change -0.187  ->  Behavior Drift
```

MALE DEVELOPER:

`dow explain` turns that into a cause-and-effect statement. The behavior changed because temperature changed. That is the exact answer we usually struggle to get from normal code history.

SCREEN (`dow eval golden`):

```
metric                golden
captures_order        1.000
greets                1.000
names_dish            1.000
states_price          1.000
confirms_spice        1.000
recommends_special    1.000
suggests_pairing      1.000
avg_response_length   78.830
```

MALE DEVELOPER:

And `dow eval` shows the custom metrics that matter for this chatbot. These are regular Python evaluators, so the team can define their own quality checklist.

SCREEN (`dow tree`):

```
chatbot - behavior evolution
v1 -> v2 -> v3 -> v4 -> v5
                  |
                  -> v6 release
```

MALE DEVELOPER:

Finally, `dow tree` shows how the chatbot evolved, including the release branch. The same tree can be exported as Mermaid for documentation.

4. Wrap-up (2:35-3:00)

FEMALE DEVELOPER:

This solves the problem I had. I can track every chatbot version, see which one was good, compare behavior, explain the cause of drift, and keep a release candidate without losing the experiment history.

MALE DEVELOPER:

That is the core workflow: run, compare, explain, evaluate, tag, and inspect the tree. `dow` gives the team a behavior history for the chatbot, not just a code history.

FEMALE DEVELOPER:

And because the demo uses the local chatbot code in demo/, we can record the UI and CLI without calling an external model.

MALE DEVELOPER:

Exactly. `dow` makes chatbot behavior versioned, measurable, and explainable.

END CARD:

```
dow
Version control for AI behavior
Track prompts, settings, outputs, drift, evaluations, and lineage.
```

Recording Notes

- Start with both developers seated at a table and the UI already visible.
- In the video, call the project simply **chatbot**. Avoid giving the chatbot a separate product name.
- The code used for the demo lives in demo/. The demo currently uses the restaurant chatbot implementation in demo/chatbot.py, custom evaluators in demo/evals.py, and the guided script in demo/run_demo.py.
- The first half should show the UI: version list, version details, comparison, tags, drift, stability, and evaluation scores.
- The second half should show the CLI: `run`, `history`, `compare`, `explain`, `eval`, and `tree`.
- Keep the language direct and product-focused. No cinematic setup, no quirky jokes, no special effects.
- Target runtime is 3 minutes. Keep UI shots readable but brief; hold the `compare` and `explain` screens slightly longer because they show the main value of `dow`.